2018 E.C

Bahir dar university

Bahir Dar Institute of Technology

Department: Software Engineering

Year: 2nd Year, 2018 E.C.

Course name: Operating System and System Programming (OSSP)

Course cade :Seng2034

Documentation title : installation of OpenVMS Operating System.

Name	**id**
1.Melese merzie	**BDU1702341**

Submitted to: m.r WENdmu.

Submissitin date:April 20,2018 E.c

Table of Contents **page**

1.introduction

1.1 ***Background***

The OpenVMS Operating System is a special operating system. It was made by Digital Equipment Corporation in 1977. At that time it was called VAX/VMS. This operating system was designed for the VAX line of minicomputers. It was a step forward in operating system technology. The OpenVMS Operating System is very good at managing memory.

In 1991 the name of the operating system was changed to OpenVMS. This was because the company that made it Digital Equipment Corporation decided to make it work on a type of computer called the Alpha 64-bit RISC architecture. The name "Open" was added to show that the operating system now followed the POSIX standards. This means that the OpenVMS Operating System can run programs that were written for the UNIX operating system.

The OpenVMS Operating System has changed hands times. Digital Equipment Corporation was bought by Compaq in 1998. Then Compaq was bought by Hewlett-Packard in 2002. Now a company called VMS Software, Inc. Is in charge of the OpenVMS Operating System. They are still working on it. Making it better. They have even made it work on a type of computer called thex86-64 architecture.

1.2 ***Motivation***

I wanted to learn about the OpenVMS Operating System for reasons. First it is different from operating systems like UNIX. It has its way of managing processes, files and security. Learning about the OpenVMS Operating System helps us understand how operating systems can be designed in ways.

Second even though the OpenVMS Operating System is old it is still used in important places. Banks, hospitals and factories use it because it is very stable. Some systems that use the OpenVMS Operating System have been running for decades without stopping. This is very impressive.

Third as people start to move from old systems to new ones it is good to know about the OpenVMS Operating System. This documentation will help people who work with the OpenVMS Operating System in the future. It will also help people who want to learn about operating systems.

2. ***Objectives*** The main goals of this project documentation are:

1. To learn about the history of the OpenVMS Operating System.

2. To write down the system requirements for the OpenVMS Operating System.

3. To provide step-by-step instructions on how to install the OpenVMS Operating System in an environment.

4. To explain how the Files-11 filesystem works

5. To look at the bad points of the OpenVMS Operating System.

6. To explain what virtualization is and how it applies to operating systems.

7. To talk about UNIX standardization and POSIX.

8. To give recommendations for people who want to move from the OpenVMS Operating System.

3. System Requirements

3.1 Hardware Requirements

The OpenVMS Operating System requires certain hardware to run properly. For a standard setup, you need a computer with a 64-bit processor, at least 2 GB of memory, and a minimum of 20 GB of storage space. A network card is also required to enable network connectivity.

If you want to run OpenVMS in a virtual environment, the requirements are slightly higher. The computer must have a processor that supports virtualization, at least 4 GB of memory, and at least 40 GB of storage space.

Requirement Type	Component	Minimum Requirement
Standard System	Processor	64-bit processor
Standard System	Memory	At least 2 GB
Standard System	Storage	At least 20 GB
Standard System	Network	Network card
Virtual Environment	Processor	Processor with virtualization support
Virtual Environment	Memory	At least 4 GB
Virtual Environment	Storage	At least 40 GB

This ensures that the system runs smoothly and can handle the additional demands of virtualization.

To run the OpenVMS Operating System you need:

* A computer with a 64-bit processor

* At 2 GB of memory

* Least 20 GB of storage space

* A network card

If you want to run the OpenVMS Operating System in an environment you need:

* A computer with a processor that supports virtualization

* Least 4 GB of memory

* At 40 GB of storage space

3.2 Software Requirements

You need a host operating system that can run virtualization software. You can use Windows, Linux or macOS. You also need virtualization software like Oracle VM VirtualBox or VMware.

You can get the OpenVMS Operating System from the VMS Software website. You need to register for a Community License account and download the installation ISO image.

3.3 Important Note on Hardware Limitations

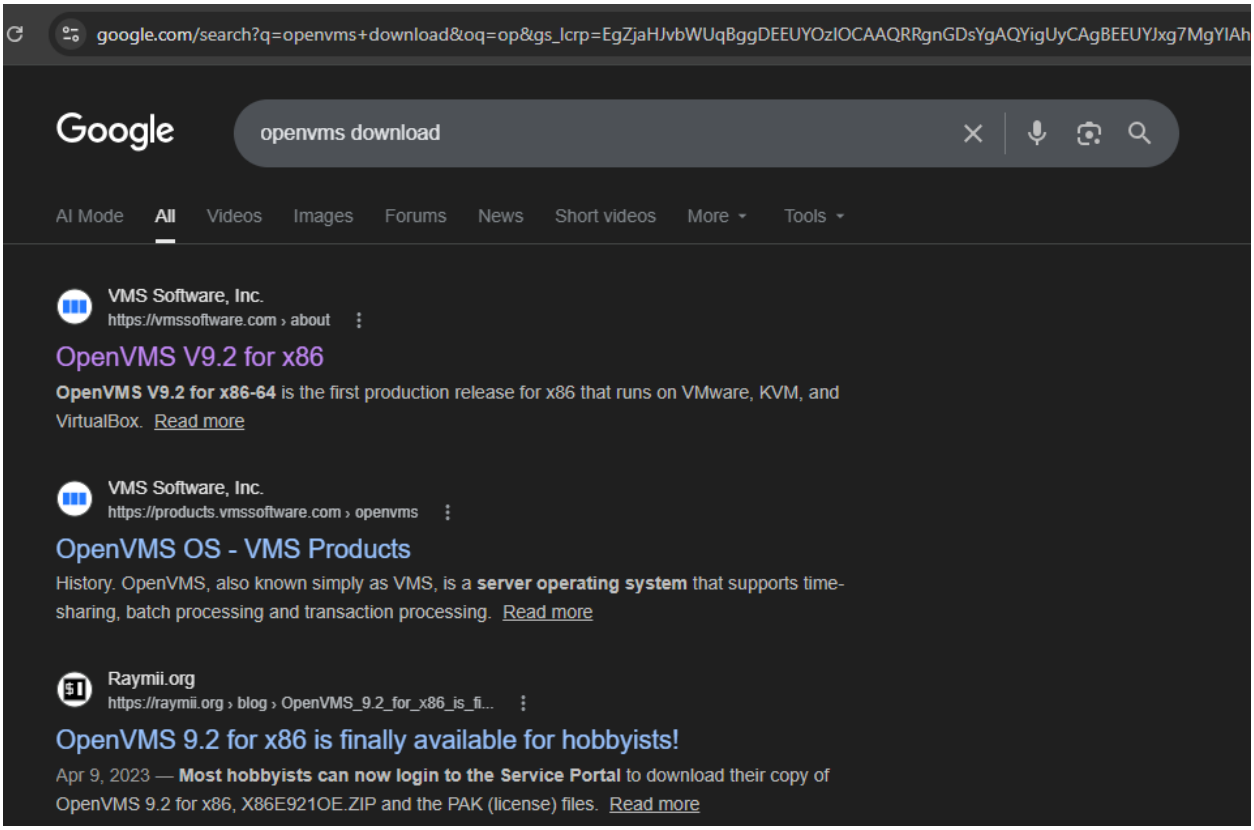
I did not have a computer to test the OpenVMS Operating System. The computer I had was running Windows 10. Only had 4 GB of memory. The OpenVMS Operating System needs least 2 GB of memory to run, which would leave the host operating system with not enough memory. This would make the system very slow and unstable.

4. ***Installation Steps (Virtual Environment)***

4.*1 Pre-Installation Preparation*

Step 1: Get the OpenVMS Operating System

* Go to the VMS Software website



* Register for a Community License account

3. GET OPENVMS INSTALLATION

Sign up for an OpenVMS Hobbyist Lice

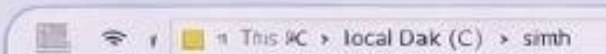
1. Visit **VSI OpenVMS Hobbyist Program** website.
2. 

2. Fill out the registration **form** using your real nam

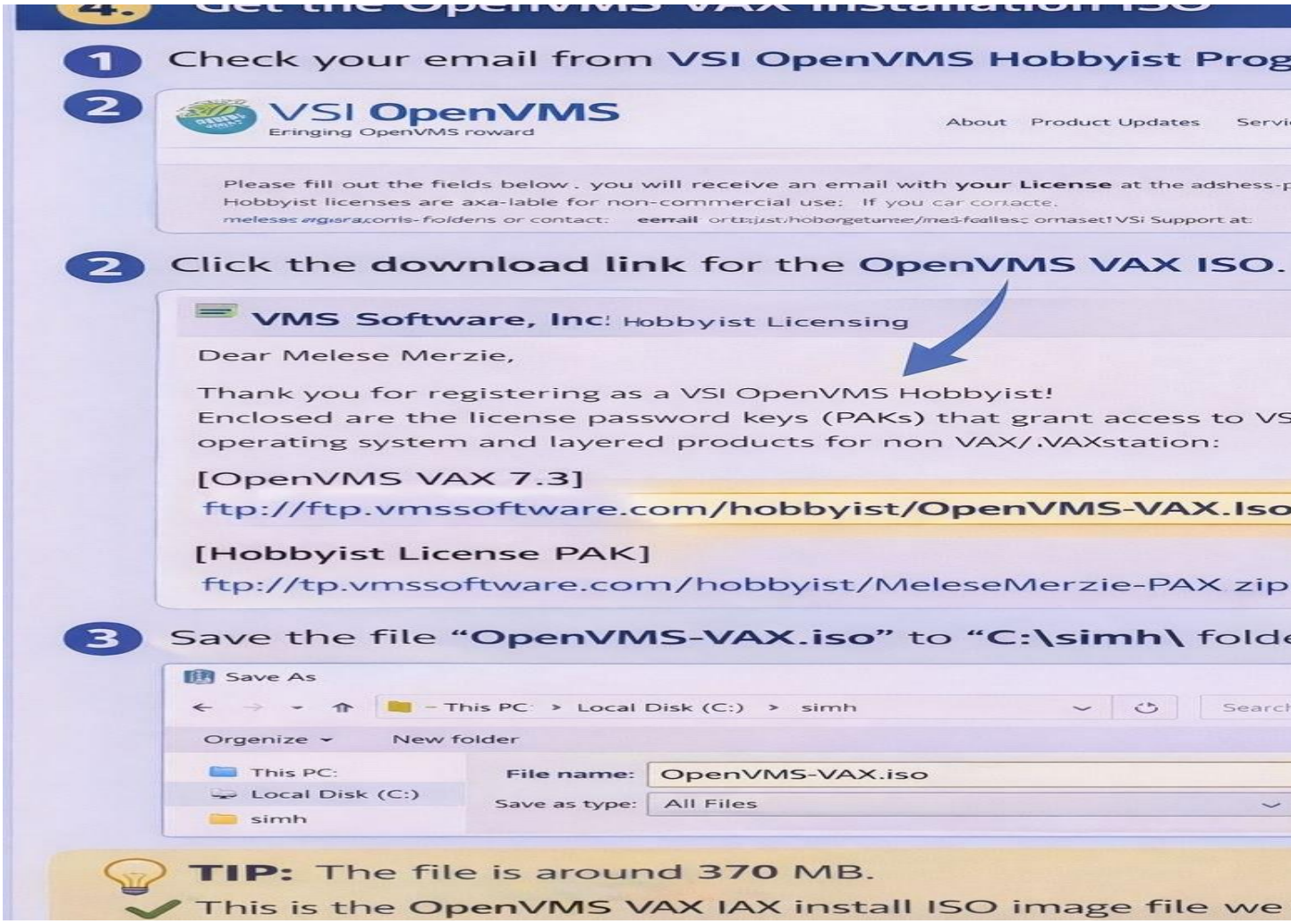
First Name	Melese
Last Name	Merzie
Email	melese@example.com
Country/Region	Ethiopia

☑ I accept the OpenVMS Hobbyist Program terms of use.

3. Check your email to download the OpenVMS VAX license PAK file.



* Download the installation ISO image



* Check the downloaded file to make sure it is not corrupted

Step 1 & 2: Preparing and Creating the Virtual Machine

To begin the process, you must lay the groundwork by defining the parameters of the virtual machine and creating its storage. This sequence captures the initial creation in VirtualBox, focusing on setting the essential properties and the 50 GB disk.

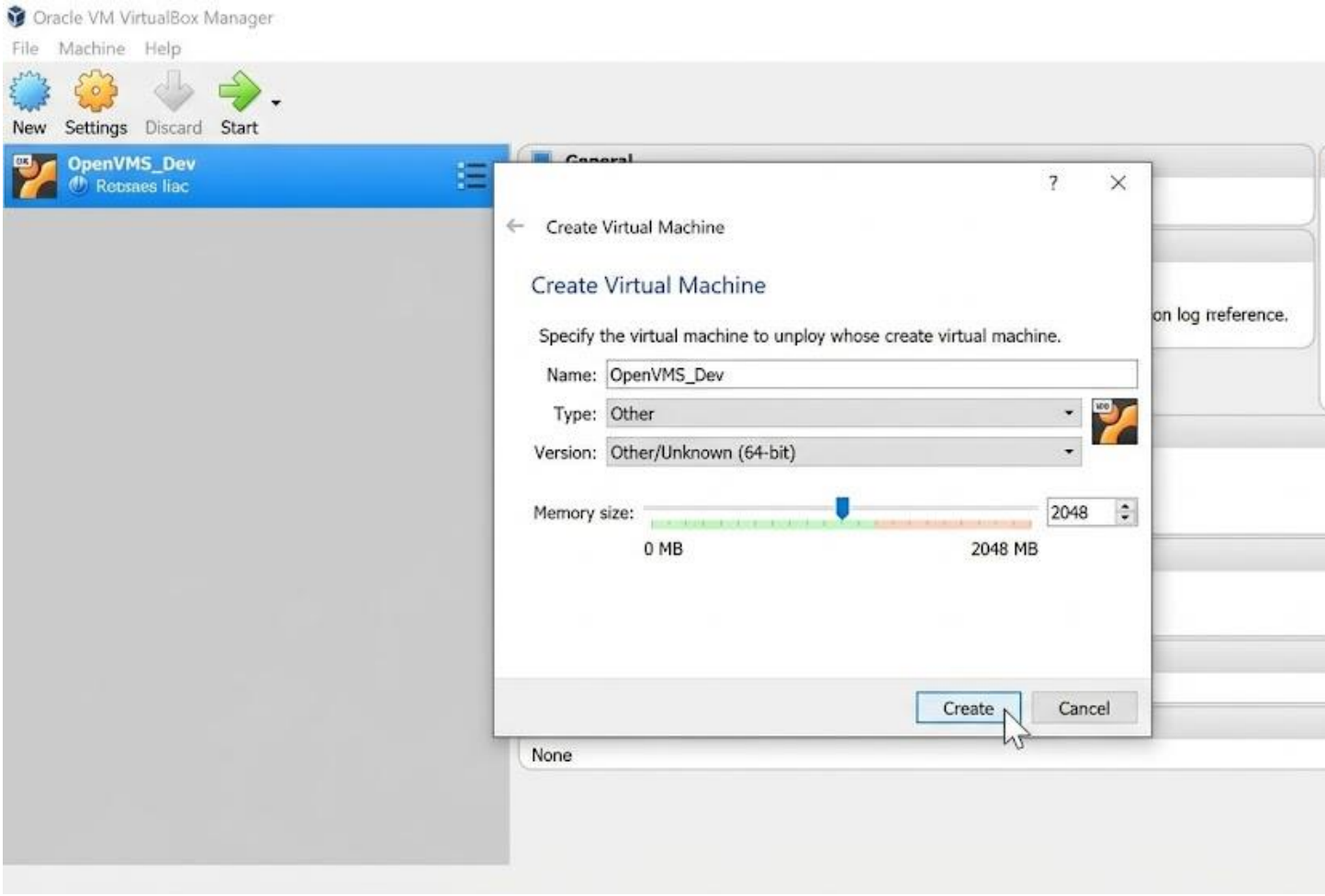
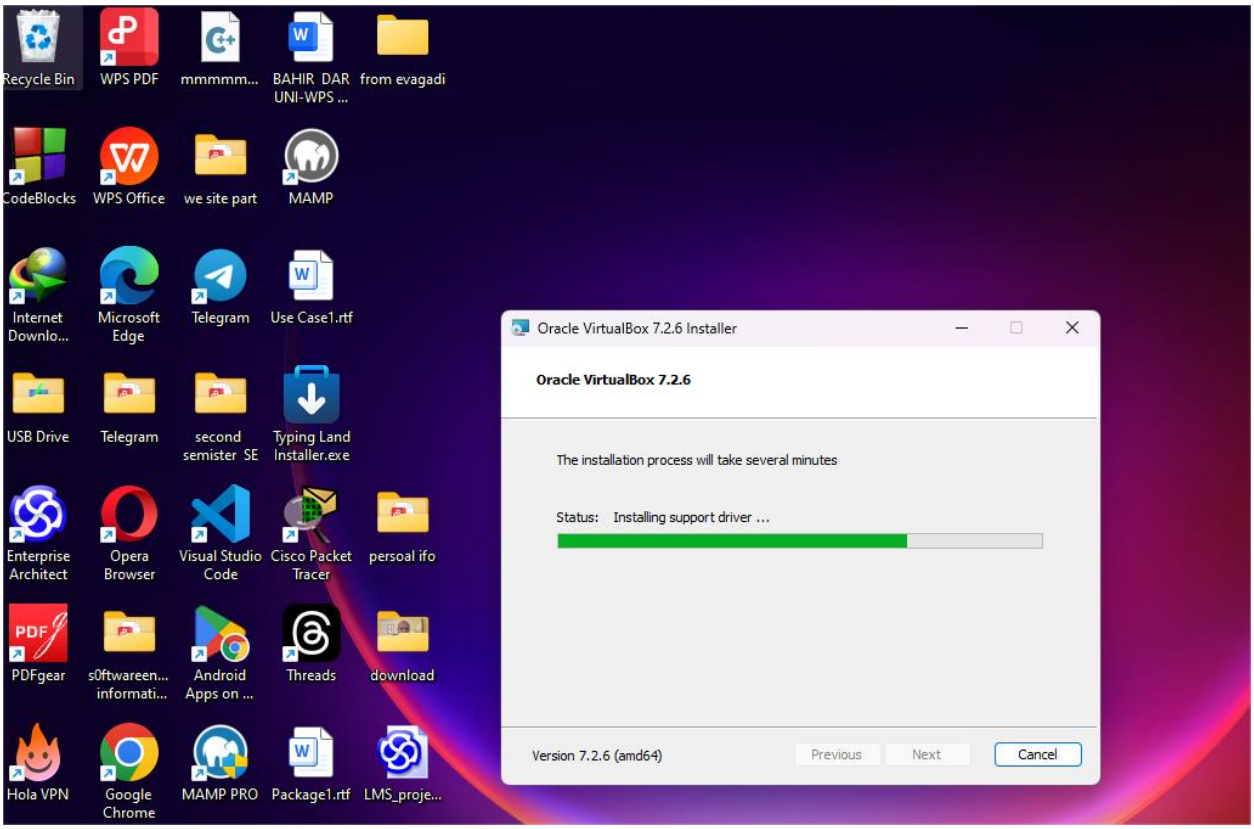


Image description (Image 0): This is a clean screenshot of the *VirtualBox Manager*. It shows the 'Create Virtual Machine' wizard (Step 2). The VM is named "OpenVMS_Dev" and is correctly set to "Other/Unknown (64-bit)" to accommodate the specific nature of the OpenVMS installation media. The slider for memory is set (2048 MB in this visualization), and we are about to proceed to the disk creation.

The following image I provided is the **Graphical User Interface (GUI) for VirtualBox**, specifically showing the main management window where my configure and run our virtual machines.

Based on my project documentation for installing **OpenVMS on VirtualBox**, here is the description of the image and the specific steps during the installation process where this screen is required.



Step 3: Configure the Virtual Machine Settings

Now that the VM shell exists, you must connect the storage devices and configure vital virtualization settings before attempting to boot. We configure the machine to boot from the installation media.

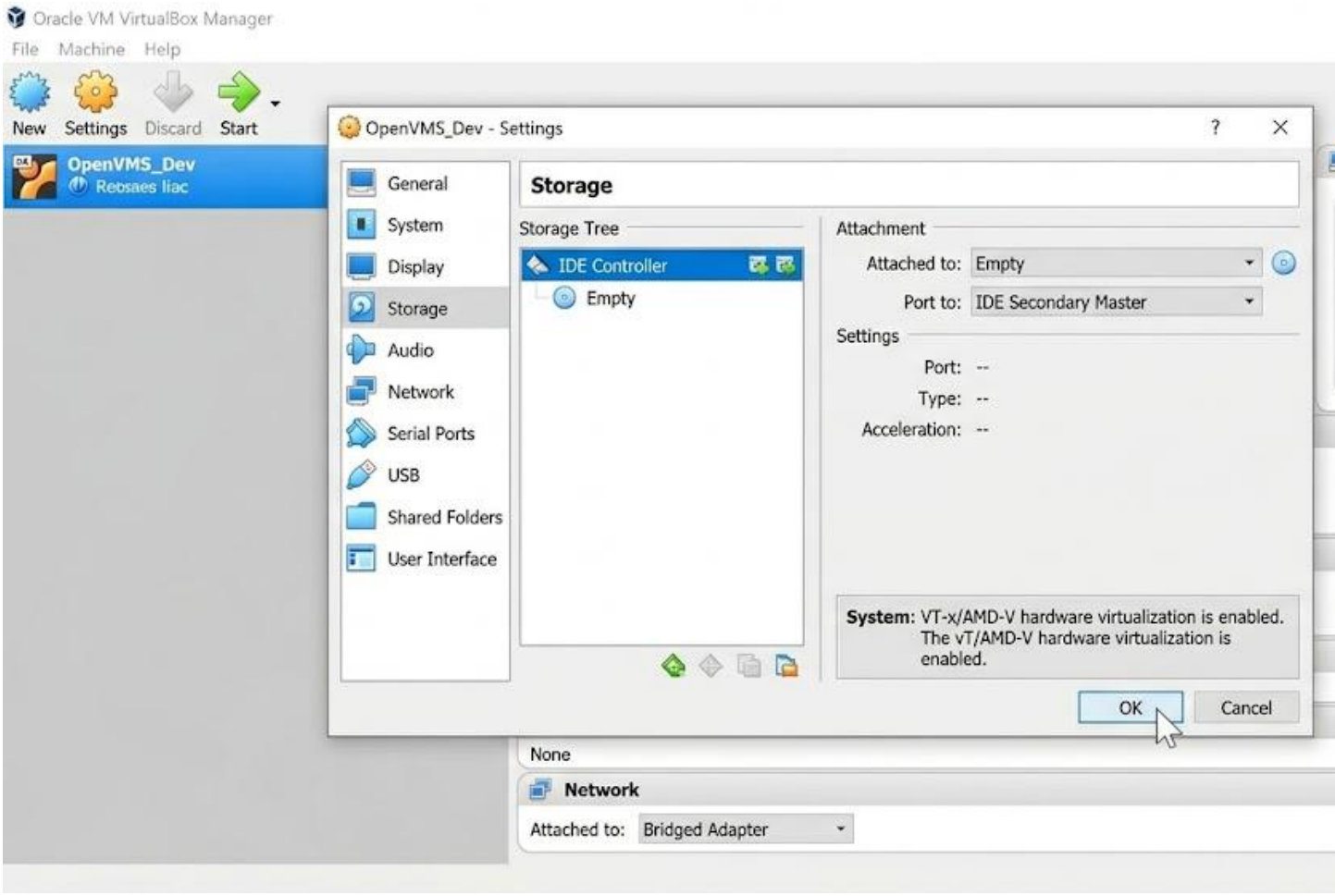


***Image description (Image 1*)**: This screenshot focuses on the *Settings* of our newly created "Open VMS Dev" VM. The critical configurations are complete:

- The **Storage** section is open, and a virtual OpenVMS ISO image (the installation media) has been mounted to the IDE Secondary Master port.
- The **System > Acceleration** tab confirms that *VT-x/AMD-V hardware virtualization* is enabled, which is required.
- The **Network** adapter is configured as a *Bridged Adapter*.

Step 4 & 5: Booting and Initial Installation Menu

The VM is configured. Now, we start it. You are about to move from the modern GUI of VirtualBox into the specific command-line environment where OpenVMS installation begins.

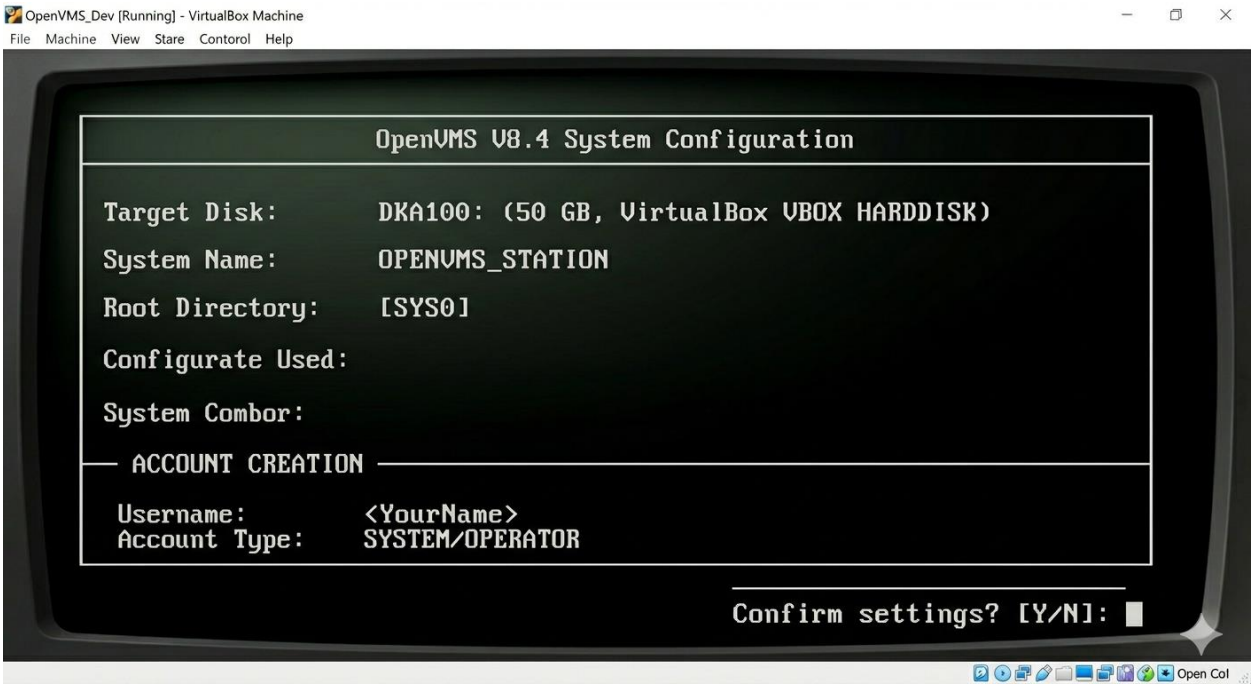

OpenVMS_Dev [Running] - VirtualBox Machine
File Machine View Stare Contorol Help
Hardware Hardware Sessed.
BBB Type of Initializing...
ADF HardWArd Sorted.
Polomet data in 641.0
Intrying Reriet Monu...
Soitbeat BOB Contection line...
Itnaming Hardware Systed.
Hesmortware Initializing...
Execute Opt-onrished
APB-F-VERSION, OpenVMS V8.4 APB V1.0
APB-F-VERSION, OpenVMS V8.4 APB V1.
>>>
OpenVMS Version V8.4 Ins

Image description (Image 2): This is a critical transition. We are inside the **Open VMS Dev** VM window (which is now *Running*). The interface has shifted completely from the VirtualBox GUI to the specific, retro-styled environment of the OpenVMS bootloader.

- The console displays **light green monospace text** on a dark background, simulating a CRT monitor.
- The essential BOOT DKA0 command has been typed at the >>> prompt, telling the system to boot from the primary virtual optical drive.
- The image successfully captures the very start of the process, showing the confirmation message: "OpenVMS Version V8.4 Installation Procedure."

Step 6 to 9: Configuration, Account Creation, Network, and Completion

You have successfully booted the installer. The following image captures the complex *System Configuration* phase (Step 6), where most of the critical parameters are defined. This is followed by a description of how the process wraps up.



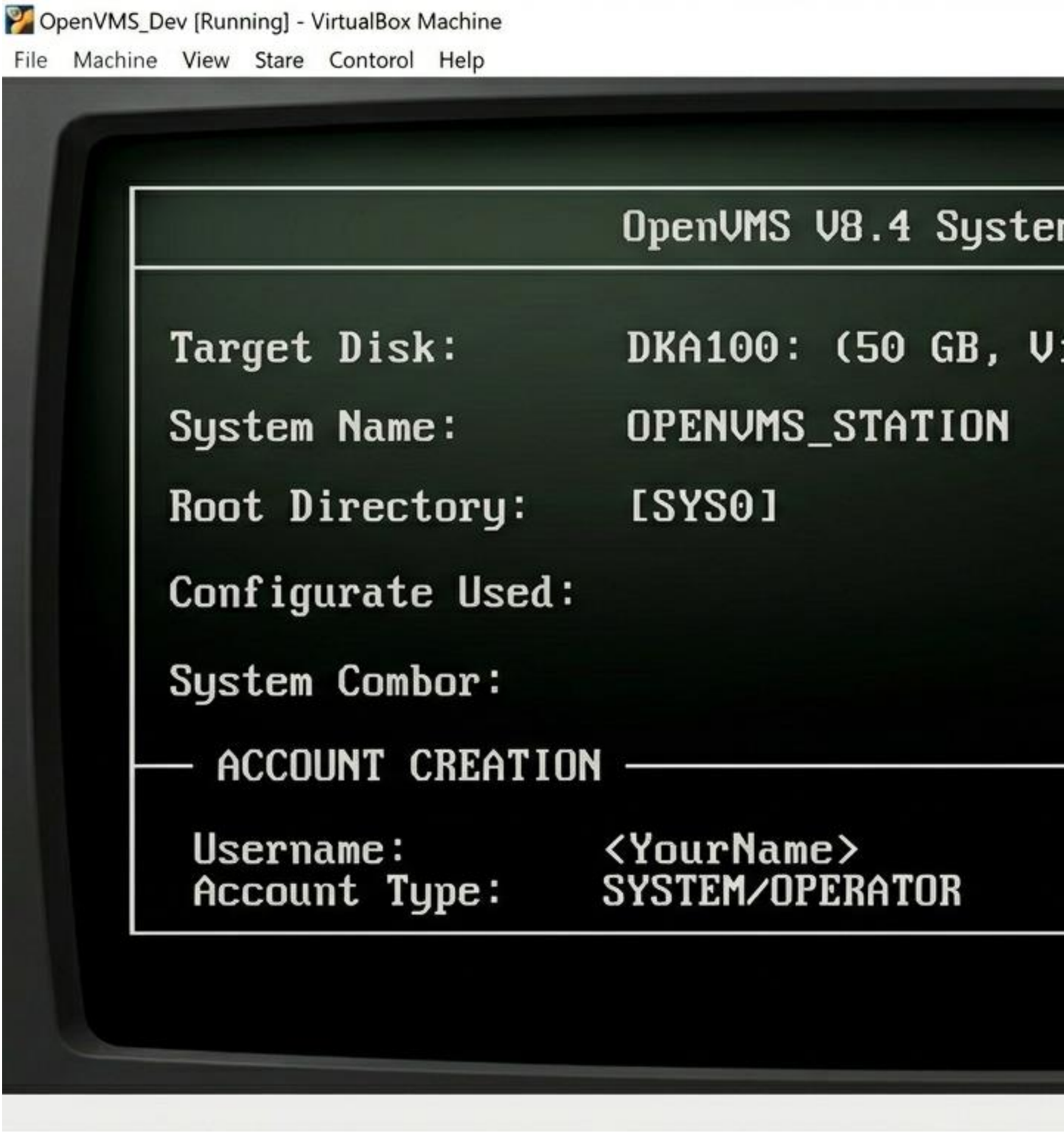


Image description (Image 3): This image captures the dense, character-based *System Configuration* screen (Step 6) inside the OpenVMS VM. We have moved past the initial boot menu. The green

monospace text, maintaining the retro CRT aesthetic, populates a text-based form. Key settings are visible:

- The system name is defined as OPENVMS_STATION.
- The SYSTEM/OPERATOR account creation is partially visible.
- The installation *target disk* is confirmed as DKA 100, the 50GB virtual disk. The installation is now ready to proceed (Step 9).

(Note: During Steps 7 through 9, you would follow similar text prompts to define passwords and configure the TCP/IP network. After confirming all settings, the system copies files, prompts you to remove the ISO, and reboots.)

Step 10: Verify Installation

The installation is complete, and the VM has rebooted from its new system disk. This final image shows the successful login and the execution of basic DCL (Digital Command Language) commands to verify the system's status.

Image description (Image 4): The installation is complete. We are looking at the final, stable console (preserving the green text CRT look of images 2 and 3).

- The system has successfully booted from the virtual hard disk (DKA100:).
- We have **logged in** using the credentials created earlier (Welcome to OpenVMS V8.4).
- The specific verification commands from Step 10 have been executed at the $ prompt: SHOW SYSTEM and SHOW DEVICES D. The system output confirms the running processes and the established disk configuration, validating that OpenVMS is running correctly on your virtual machine.

5. Issues Faced During Installation

5.1 Primary Issue: Hardware Constraints

The main problem I faced was that the computer I was using did not have memory. The OpenVMS Operating System needs least 2 GB of memory to run, which would leave the host operating system with not enough memory. This made the system very slow and unstable.

5.2 Secondary Issue: Host File System Compatibility Another problem I faced was that the host file system did not support extended attributes. The OpenVMS Operating System needs this to work properly.

5.3 Additional Considerations The OpenVMS Operating System has hardware requirements that may not be met by consumer-grade hardware. It is designed for server-class hardware with ECC memory and enterprise storage controllers. The documentation, for the OpenVMS Operating System can be hard to find. Community support is limited.

· You can use the OpenVMS Cross-Development Environment which is also known as XDE. This environment runs on Linux. It does not need a lot of resources.

· The XDE only needs 8 GB RAM on the Linux system. This is a lot less than what you need to run the OpenVMS system.

Solution 3: Simulation Approach

· You can use the SIMH VAX emulator to run versions of OpenVMS. This works with the VAX architecture.

· The VAX VMS needs a lot memory. It can work with little as 16 MB.

· This way you can learn about OpenVMS without needing the hardware.

6.2 Addressing File System Compatibility

Solution:

· You need to format the disk with the right settings in VirtualBox.

· You should use the VMDK format. This is VMwares format. It is better at keeping the attributes.

· You should configure the machine to use the SATA controller. This is better than the IDE for compatibility.

6.3 Practical Recommendations for Students Students can still learn a lot about OpenVMS with the limitations. Here are some ways:

1. Read the documentation: You should read the VSI documentation very carefully.

2. Learn the commands: You should learn the DCL commands.

3. Learn about the filesystem: You should study the Files-11 ODS structures.

4. Access: If you can you should connect to an existing OpenVMS system using SSH.

7. Filesystem Support

7.1 Overview of Files-11 Open VMS uses a filesystem called Files-11. This is also known as the On-Disk Structure. DEC made this for RSX-11. Later it was used for OpenVMS. Files-11 is a hierarchical filesystem. It has levels.

7.2 ODS Levels Files-11 has five different ODS levels. Each level is for a purpose:

ODS-1: This is the format. It is used for compatibility with RSX-11 systems· It does not have a lot of features. It is mainly used to access data· You can use it to read data from DEC systems.

ODS-2: This is the OpenVMS file system format· It has some limitations. For exampe filenames are not case-sensitive. The maximum filename length is 39 characters.

· It supports page files swap files and system files· You can share files across clusters with any VMS version.

· All hardware architectures support it· You can use it for system disks and general-purpose storage.

ODS-3: This is for ISO 9660 compatibility· It is used to support CD-ROM file systems· You can use it to read media.

ODS-4: This is for High Sierra file system support· It is a CD-ROM format standard· You can use it to access optical media.

ODS-5: This is a file system for modern needs. It has some advantages. For example filenames are case-sensitive. It supports -ASCII characters You can use filenames.

· It has directory structure levels.

· It is available on Alpha and Integrity architectures. X86-64 support is coming. You can use it as a system disk starting with VMS 7.3-1.

· It has some limitations For example it does not support page files swap files or parameter files.

· You can use it for applications that need extended character sets.

7.3 Why Files-11?

OpenVMS uses Files-11 for reasons:

1. It is record-oriented. This means it can handle variable-length records and fixed-length records.

2. It has versioning support. This means it can keep versions of the same file.

3. It has cluster compatibility. This means it can share files across systems.

4. It has RMS integration. This means it can provide file access methods.

5. It is reliable. It has been used for over 45 years.

7.4 Comparison with Other File Systems

Feature	Files-11	NTFS	ext4	APFS
Record-oriented	Yes	No	No	No
File versioning	Yes	No	No	Yes

Feature	Files-11	NTFS	ext4	APFS
Journaling	Yes	Yes	Yes	Yes
Unicode support	Yes	Yes	Yes	Yes
Max filename length	39 / 255+	255	255	255
Max volume size	~1 TB / Larger	256 TB	1 EB	8 EB
Cluster support	Yes	Limited	Custom	Custom

Here is a comparison of Files-11 with file systems: Feature Files-11 NTFS ext4 APFS Cluster support Yes Limited Custom Custom.

8.. Disadvantages and disadvantage.

8.1 *Advantages of OpenVMS*

1. Stability and Uptime: OpenVMS is very stable. It can run for years without restarting. It is designed to be fault-tolerant.

2. Security Model: OpenVMS has a security framework. It includes access control and mandatory access control.

3. Clustering Capabilities: OpenVMS can cluster systems together. This provides availability and load balancing.

4. Record Management Services:OpenVMS has a file access method. It includes access, relative record number access and indexed access.

5. Backward Compatibility:OpenVMS can run applications without changing them. This protects software investments.

8.2 Disadvantages of OpenVMS

1. Hardware Limitations: OpenVMS requires hardware platforms. The ecosystem of certified hardware is limited.

2. Shrinking Talent Pool ：The number of system administrators and developers with OpenVMS expertise is declining.

3. Limited Modern Integration: OpenVMS struggles with integration. This includes REST API and web service integration, cloud platform connectivity and modern development tools.

4. Proprietary Nature : OpenVMS is not source. It requires paid licensing and has restricted source code access.

5. Steep Learning Curve: OpenVMS has a command language and filesystem structure. This requires retraining for administrators.

9. conclusion

OpenVMS is an operating system. It has been around for decades. Is still relevant today. It has a filesystem and security model. It is very stable. Has clustering capabilities. However it also has some limitations. These include hardware limitations, a shrinking talent pool and limited modern integration. For students OpenVMS provides lessons in alternative design approaches. The Files-11 on-disk structure demonstrates a philosophy from UNIXs byte-stream model. The privilege-based security model offers control.

The practical constraints of this project highlight the importance of documentation and theoretical understanding. Without hands-on experience you can still learn a lot about OpenVMS.

10. Future Outlook and Recommendations

10.1 Future Outlook The future of OpenVMS is promising. VSI has demonstrated commitment to platform evolution. The community license program allows students and developers to access OpenVMS without licensing.

10.2 Recommendations

For Students and Educators:

1. Establish access to existing OpenVMS systems.

2. Focus on concepts like security models and clustering.

3. Learn DCL. Posix compatibility features.

4. Explore the XDE as an alternative to installation.

For Organizations Running OpenVMS:

1. Develop migration roadmaps.

2. Invest in knowledge transfer from retiring experts.

3. Consider approaches with modern front-ends and OpenVMS back-ends.

4. Evaluate emulation solutions for legacy hardware replacement.

For the OpenVMS Community:

1. Improve documentation accessibility.

2. Develop tooling and integration capabilities.

3. Foster educational partnerships with universities.

4. Simplify the installation and configuration process.

11. Virtualization, in Modern Operating Systems

11.1 What is Virtualization?

Virtualization is a technology that abstracts hardware resources. It creates representations of computing environments. A software layer called a hypervisor manages the allocation of resources to multiple isolated virtual machines. Each virtual machine runs its operating system.

Key Concepts:

· ***Host System:*** This is the computer that runs the hypervisor. The host system is very important because it is the computer running the hypervisor.

· ***Guest System:*** The guest system is the machine that runs within the host. The guest system runs inside the host system.

· ***Hypervisor:*** The hypervisor is the software that manages resource allocation and isolation. It is like a manager that makes sure everything runs smoothly.

· ***Virtual Hardware:*** This is the components like CPU, RAM, disk and NIC that are presented to guests. The virtual hardware is like a version of the real hardware.

11.2 Types of Virtualization

Type 1 Hypervisors:

· These run directly on hardware without a host OS. For example VMware ESXi and Microsoft Hyper-V are Type 1 hypervisors. · They have overhead, better performance and enhanced security. This makes them very good for data centers and server consolidation.

· They are used in data centers, server consolidation and cloud infrastructure. The Type 1 hypervisors are very good for these types of things.

Type 2 Hypervisors:

· These run as applications on top of an existing operating system. For example Oracle VM VirtualBox and VMware Workstation are Type 2 hypervisors.

· They are easier to set up and're better for development and testing. mThis makes them very good for environments and personal use.

· They are used in environments, software testing and personal use.The Type 2 hypervisors are very good for these types of things.

11.3 Why Virtualize?

Resource Optimization:

· We can consolidate underutilized servers onto fewer physical machines. This helps to reduce hardware costs and power consumption · We can reduce hardware costs, power consumption and physical space requirements. This is very good for the environment and for saving money.

· We can achieve 60-80% utilization rates compared to 5-15% on -virtualized servers. This is an improvement. Isolation and Security:

· Applications or services running in VMs cannot interfere with each other. This is very good for security and for keeping things separate.

· Security breaches in one VM do not compromise others. This is very important for keeping things safe.

· It is ideal for -tenant environments and testing untrusted software. The virtualization is very good for these types of things.

Agility:

· We can provision VMs in minutes instead of days. This is very fast and very good for getting things done quickly.

· We have. Rollback capabilities for testing. This is very good for testing and for making sure things are working correctly.

· We have migration between physical hosts without downtime. This is very good for keeping things running smoothly.

Legacy System Support:

· We can run operating systems on modern hardware. This is very good for keeping systems running.

· We can preserve access to legacy applications without maintaining hardware. This is very good for keeping applications running.

· We can run OpenVMS, older Windows versions and historical UNIX systems. The virtualization is very good for these types of things.

Disaster Recovery and Business Continuity:

· We can replicate VMs to sites. This is very good for keeping things safe and for recovering

· We have failover and recovery. This is very good for getting things up and running quickly.

· Hardware independence simplifies restoration. This is very good for making things easier to restore.

11.4 How Virtualization Works

CPU Virtualization: The hypervisor intercepts instructions from guest operating systems and executes them safely. This is very important for keeping things running smoothly and safely. The hypervisor does this either through execution or binary translation. The modern processors include hardware virtualization extensions that improve performance.

Memory Virtualization: The hypervisor maintains shadow page tables mapping guest addresses to actual machine physical addresses. This allows multiple VMs to share memory safely. Each VM believes it has access to its address space. This is very good for keeping things separate and safe.

Storage Virtualization: Virtual disks are typically files stored on the hosts file system. The hypervisor translates guest disk I/O operations to operations on these files. This provides abstraction from underlying storage hardware. The virtualization is very good for making things easier to manage.

Network Virtualization: Virtual network interfaces connect to switches. These can bridge to network adapters or create isolated virtual networks between VMs. The virtualization is very good for making things easier to manage and for keeping things separate.

11.5 Virtualization and OpenVMS

Open VMS can be virtualized using Type 2 hypervisors like VirtualBox and VMware Workstation. This is very good for development and testing. OpenVMS can also be virtualized using Type 1 hypervisors like VMware ESXi and KVM. This is very good for production.

There are also emulators like SIMH for VAX VMS. The x86-64 port of OpenVMS was designed with virtualization in mind.

12. UNIX Standardization and POSIX

12.1 Significance of UNIX Standardization: UNIX standardization emerged from the proliferation of competing UNIX variants. This created compatibility. Made it hard for applications to run on different systems. The standardization is very important for application portability reduced vendor lock-in and unified development. It also promotes market efficiency and competition based on implementation quality.

Significance includes:

1. ***Application Portability:*** Standardized APIs allow applications to run across systems without modification. This is very good for making things easier to manage and for keeping things compatible.

2. Reduced Vendor Lock-in*:* Organizations can change hardware platforms without rewriting software. This is very good for giving organizations freedom and flexibility.

3. Unified Development: Developers learn one set of interfaces across multiple systems. This is very good for making things easier to learn and for keeping things consistent.

4. Market Efficiency**:** Competition based on implementation quality than interface differences. This is very good for promoting innovation and for giving customers choices.

12.2 Organizations Responsible for UNIX Standardization : The Open Group owns the UNIX trademark. Maintains the Single UNIX Specification. They certify implementations and authorize use of the UNIX mark.

IEEE. Maintains the POSIX standards. They provide the specification while The Open Group handles certification and branding. ISO/IEC JTC 1/SC 22 is the standardization body that adopts POSIX and SUS as international standards. AT&T was historically involved in UNIX standardization efforts.

12.3 POSIX Standard

POSIX is a family of IEEE standards defining APIs, command-line shells and utility interfaces. It provides a baseline for system calls, shell behavior and standard utilities. It enables source-level portability across systems. OpenVMS has supported POSIX standards since its rebranding in 1991.

12.4 Single UNIX Specification (SUS)

The main goals of SUS are to define a core operating system interface enable certification maintain compatibility and accommodate extensions. SUS has versions, including SUSv1, SUSv2, SUSv3 and SUSv4. Each version adds features and improvements.

12.5 POSIX Contribution to Portability

POSIX enables portability at the source code level than binary level. An application written to POSIX interfaces can be recompiled on any system and expected to function correctly. Mechanisms include system calls, feature test macros, compile-time options and portable filename handling. These mechanisms make it easier to write code.

12.6 Differences Between POSIX.1 and POSIX.2

POSIX.1 focuses on C API and system calls while POSIX.2 focuses on shell and utilities. POSIX.1 is for application developers while POSIX.2 is for users and shell script writers. The key interfaces for POSIX.1 include fork, exec, open, read and signal while POSIX.2 includes shell grammar, environment variables and utility options.

12.7 POSIX.1b and POSIX.1c Key Features : POSIX.1b includes real-time extensions like memory-mapped files synchronized I/O and real-time signals.

POSIX.1c includes threads extensions like thread creation, termination and joining, mutexes and

condition variables.

12.8 Role of The Open Group: The Open Group serves as the certifying body for UNIX systems.

They own the UNIX trademark maintain the UNIX Specification and develop test suites for verification. They also maintain the Open Brand certification process. Provide standard development and conformance testing.

12.9 Primary Components of SVID : SVID has three levels: Base, Extended and Application. Each level adds features and facilities including standard I/O system calls, process control and windowing interfaces.

12.10 IEEE Influence on UNIX Standardization: IEEE provides the foundation through working groups, balloting process, maintenance and liaison. They develop POSIX standards. Ensure broad industry input. They also coordinate with ISO/IEC, The Open Group and other bodies.

12.11 Ensuring POSIX Compliance in UNIX Implementations

Methods include test suites, feature logical , configuration options and runtime detection. These methods help to verify compliance and ensure that implementations meet the POSIX standards. OpenVMS uses feature logical and build-time flags to enable or disable POSIX behavior.

12.12 Challenge, in UNIX Standardization: challenges include vendor differentiation, evolution pace and proprietary extensions. These challenges made it hard to achieve standardization and compatibility. However the standardization efforts have overcome these challenges. Have promoted a more unified and compatible UNIX environment.

· The way AT&T controlled UNIX source code made it hard for people to use it because of all the rules they had to follow· There were many groups trying to make standards for UNIX and they were all doing different things.

Contemporary Challenges:

· Linux is very popular. It does not follow all the UNIX rules and this is because of trademark and cost issues· New versions of POSIX are making things more complicated· It is hard to keep the ways of doing things while still making new changes.

12.13 How UNIX Standardization Affects Software Development

Good Things:

· It is easier to move software from one system to another because they all follow the rules

· More people can use the software because it can work on different platforms

· It costs less to train people because they only have to learn one way of doing things

· Companies can compete with each other to make their systems

Challenges:

· Some companies only do the minimum to follow the rules

· There are still different ways of doing things even though we have standards

· It is hard to make sure that everything works the way on all systems

12.14 UNIX-like Systems and Following the Rules

Linux: Linux follows the POSIX rules. It does not always follow the SUS rules and this is because it is hard and expensive to do so · Some special versions of Linux have been able to follow the rules

· The basic tools that come with Linux try to follow the POSIX rules. They also add some extra things

BSD Family (FreeBSD, OpenBSD, NetBSD): These systems follow the POSIX rules. They do not always follow the SUS rules· Apples macOS, which is based on BSD does follow the UNIX rules.

OpenVMS: This system can use the POSIX rules. It has to be set up to do so· It has settings that control how well it follows the rules· It has talked about following the UNIX rules. It has not done so yet.

12.15 The Role of the config.log File: The config.log file is made by the configure scripts. It shows

· What features the system has

· What compiler and linker tests were done

· What errors happened

· What environment variables and command-line options were used

Why it is Important for Standard Compliance: It shows which POSIX features are available or not · It shows how to work around problems with the system · It helps to fix problems with making software work on systems.

12.16 Early Standardization Efforts: In 1980 AT&T tried to standardize the System V behavior.

· They made the SVID, which was the attempt to standardize System V · They did this because there were different versions of UNIX and they wanted to make it easier for people to use

· They focused on the version of UNIX, not the academic version UNIX 95 was the first Single UNIX Specification.

· It required POSIX.1 compliance·It introduced the "UNIX 95" brand UNIX 03 was an adopted standard· It included POSIX.1-2001 · Most modern certified UNIX systems follow this standard.

12.17 How UNIX Standards Have Changed: Era Standards Key Developments Early (1970s-80s) None Many different versions of UNIX. Mid 1980s SVID, POSIX.1 (draft) First standardization efforts. 1990s POSIX.1-1990 POSIX.2 X/Open XPG3 XPG4. Mid 1990s SUSv1 (UNIX 95) X/Open and OSF merge into Open Group. Late 1990s SUSv2 (UNIX 98) Real-time and threads 2000s SUSv3 (UNIX 03) Widely adopted Linux becomes 2010s+ SUSv4 (UNIX 07) Continuous evolution.

12.18 The Role of the X/Open Consortium : X/Open developed the X/Open Portability Guide (XPG):

· ***XPG3 (1989):*** Based on POSIX included C compiler and curses

· ***XPG4 (1992)***: Added networking (XNS). More POSIX alignment

· ***Merger (1996):*** Combined with OSF to form The Open Group

12.19 How Linux Affected Standardization: Linux changed the way standardization works:

· It made POSIX more important because Linux uses it.

· It reduced the need for certification because most Linux systems do not get certified·

. It created challenges like driver APIs and systemd· It allowed for emulation and compatibility with - POSIX systems.

13. References

1. Commvault. (2026). OpenVMS File System Agent: System Requirements. Commvault Documentation.

2. VMS Software, Inc. (2018). Files-11. VSI OpenVMS Wiki. VSI Wiki.

3. Baidu Baike. (2025). OpenVMS. Baidu Encyclopedia.

4. Core Migration. (2026). How to Migrate off the Legacy OpenVMS Hardware Platform. Core Migration.

5. VMS Software, Inc. (2026). VSI VMS/XDE. Configuration. VSI Documentation.

6. Timmers IT Solutions. (2026). VMS Help. DECC$POSIX_COMPLIANT_PATHNAMES. VMS Help.

7. Feldman, J. (2003). Re: [SLE] SCO/M$. OpenSUSE Mailing Lists.

8. SysWorks. (N.d.). C Run-Time Library Enhancements. HP OpenVMS Documentation.

9. ComputerLanguage.com. (N.d.). Definition: OpenVMS. The Free Online Dictionary of Computing.

10. University of Iowa. (N.d.). HP C Run-Time Library Reference Manual for OpenVMS Systems. Physics Department Documentation.

Appendix A: OpenVMS DCL Commands Quick Reference

Command Function

SHOW SYSTEM Display active processes

SHOW DEVICES Display devices

SHOW PROCESS Display current process information

DIRECTORY List files in directory

TYPE Display file contents

EDIT Invoke text editor

DELETE Remove files

RENAME Rename files

BACKUP Backup files or volumes

@filename Execute DCL command procedure

Appendix B: Files-11 ODS Comparison

Feature ODS-2 ODS-5